

RAPPORT DE STAGE

Deuxième Année

Développement d'une application desktop JavaFX de gestion de stock — Restaurant Kot Robert

Entreprise d'accueil :

Kot Robert — Restaurant

Île Maurice

Réalisé par : FANEKENA Diary

Tuteur de stage : Responsable du Restaurant Kot Robert

Année académique : 2025 – 2026

Durée du stage : 6 semaines

Introduction

Dans le cadre de ma première année de BTS SIO option SLAM au sein de MCCI Business School, j'ai effectué un stage de six semaines au restaurant Kot Robert, situé à l'Île Maurice.

Le restaurant Kot Robert est un établissement de restauration mauricien proposant une cuisine variée, avec une offre de plats sur place et à emporter (take-away). Face à la croissance de l'activité, la gestion manuelle des stocks alimentaires et des fournitures devenait de plus en plus difficile et source d'erreurs.

Dans ce contexte, j'ai été chargé de développer une application desktop de gestion de stock, permettant au personnel de suivre en temps réel les niveaux de stock, d'enregistrer les entrées et sorties de produits et d'être alerté automatiquement en cas de stock faible.

Ce rapport présente le déroulement de ce projet, de l'analyse des besoins jusqu'à la livraison de l'application.

Sommaire

Introduction	2
Sommaire	3
Partie 1 — Contexte et Cadre du Stage	5
A. Présentation de l'entreprise	5
B. Besoin du client — Problématique	5
Contexte et problème identifié.....	5
Objectif du projet.....	5
C. Mission	5
D. Démarche projet	5
Plan de travail — Diagramme de Gantt.....	5
Outils et technologies utilisés	6
Partie 2 — Analyse et Orientations de Solution.....	7
A. Analyse du projet	7
B. Orientations de solution proposées	7
Solution 1 — Application desktop Java/JavaFX (retenue)	7
Solution 2 — Application web (PHP/MySQL)	7
Solution 3 — Logiciel de caisse avec gestion de stock intégrée.....	7
Partie 3 — Conception	9
A. Architecture MVC	9
B. Diagramme de classes.....	9
C. Modèle de base de données	9
D. Charte graphique	9
Partie 4 — Développement	11
A. Outils de développement.....	11
B. Fonctionnalités développées	11
1. Tableau de bord (DashboardView).....	11
2. Gestion des produits (StockView)	11
3. Entrées et Sorties (MouvementView)	11
4. Alertes stock (AlertesView)	11
5. Historique (HistoriqueView).....	12
6. Authentification et rôles	12
C. Données préremplies	12
Partie 5 — Recettage, Déploiement et Documentation	13
A. Recettage — Tests fonctionnels.....	13
B. Mise en place	13
Déploiement en local	13
Prérequis d'installation	13
C. Documentation technique	13

D. Formation.....	14
Conclusion	15
Annexes.....	16
Annexe 1 — Structure du projet.....	16
Annexe 2 — Captures d'écran de l'application.....	16
Annexe 3 — Bibliographie et Webographie	16
Annexe 4 — Glossaire.....	16

Partie 1 — Contexte et Cadre du Stage

A. Présentation de l'entreprise

Le restaurant Kot Robert est un établissement de restauration implanté à l'Île Maurice. Il propose une cuisine mauricienne et asiatique variée : plats à base de viandes, fruits de mer, riz, nouilles, ainsi qu'une large gamme de boissons. Le restaurant propose également un service de vente à emporter (take-away) très apprécié de sa clientèle.

L'établissement gère un stock de plus de 50 références de produits alimentaires répartis en plusieurs catégories : viandes, fruits de mer, féculents, légumes, boissons, condiments et fournitures d'emballage.

B. Besoin du client — Problématique

Contexte et problème identifié

Lors de l'entretien initial avec le responsable du restaurant, les problèmes suivants ont été identifiés :

- Gestion des stocks entièrement manuelle sur cahier ou fichiers Excel non centralisés
- Impossible de connaître en temps réel le niveau exact de chaque produit
- Ruptures de stock fréquentes faute d'alertes automatiques
- Aucun historique fiable des entrées et sorties de produits
- Pertes financières dues aux produits expirés non détectés à temps
- Difficulté à calculer la valeur totale du stock

Objectif du projet

Développer une application desktop intuitive permettant au personnel du restaurant de gérer efficacement son stock, d'enregistrer tous les mouvements de produits et d'être alerté automatiquement dès qu'un produit passe sous son seuil d'alerte.

C. Mission

- Analyser les besoins du restaurant et définir les fonctionnalités de l'application
- Concevoir la base de données PostgreSQL
- Développer l'interface graphique avec JavaFX
- Implémenter la logique métier (DAO pattern, architecture MVC)
- Peupler la base avec les 54 produits réels du restaurant
- Tester et livrer l'application au responsable

D. Démarche projet

Plan de travail — Diagramme de Gantt

Tâche	S1	S2	S3	S4	S5	S6
Analyse des besoins	✓					
Conception BDD & maquettes		✓				
Développement interface JavaFX			✓	✓		
Intégration PostgreSQL (DAO)				✓		
Tests & corrections					✓	
Livraison & formation						✓

Outils et technologies utilisés

- Langage de programmation : Java 17
- Framework UI : JavaFX 21
- Base de données : PostgreSQL
- Gestionnaire de dépendances : Maven
- IDE : IntelliJ IDEA
- Administration BDD : pgAdmin
- Modélisation : StarUML
- Gestion de projet : Trello

Partie 2 — Analyse et Orientations de Solution

A. Analyse du projet

Il s'agissait d'un nouveau projet développé intégralement à partir de zéro. Aucun outil informatique n'était utilisé pour la gestion du stock. L'analyse a permis d'identifier les fonctionnalités essentielles :

- Tableau de bord avec KPIs (total produits, alertes, valeur stock, mouvements du jour)
- Gestion CRUD des produits (ajout, modification, suppression, recherche, filtrage)
- Enregistrement des entrées et sorties avec motif et date/heure
- Système d'alertes automatiques pour les produits sous le seuil
- Historique complet de tous les mouvements filtrables
- Calcul automatique de la valeur totale du stock

B. Orientations de solution proposées

Trois solutions ont été proposées au responsable, du moins cher au plus cher :

Solution 1 — Application desktop Java/JavaFX (retenue)

Développement d'une application desktop JavaFX connectée à une base PostgreSQL locale.

- Coût estimé : 0 € (technologies open source)
- Avantages : fonctionne sans internet, rapide, interface riche, données locales sécurisées
- Inconvénients : accessible uniquement sur le poste installé
- Statut : Solution retenue et validée par le responsable

Solution 2 — Application web (PHP/MySQL)

Développement d'une application web accessible depuis n'importe quel appareil du réseau.

- Coût estimé : 0 à 100 €/an selon hébergement
- Avantages : accessible depuis tablette ou smartphone en cuisine
- Inconvénients : nécessite un serveur, développement plus long
- Statut : Non retenue, envisageable à terme

Solution 3 — Logiciel de caisse avec gestion de stock intégrée

Acquisition d'un logiciel professionnel tout-en-un (ex : Lightspeed, Square POS).

- Coût estimé : 500 € à 2 000 €/an
- Avantages : solution complète avec caisse, facturation, stock

- Inconvénients : coût élevé, fonctionnalités superflues pour les besoins actuels
- Statut : Rejetée en raison du budget

Partie 3 — Conception

A. Architecture MVC

L'application suit une architecture MVC (Modèle-Vue-Contrôleur) claire :

- Modèle (model/) : classes Produit et Mouvement représentant les entités métier
- Vue (ui/) : vues JavaFX — DashboardView, StockView, MouvementView, AlertesView, HistoriqueView
- DAO (dao/) : couche d'accès aux données — ProduitDAO, MouvementDAO, DatabaseManager
- Utilitaires (util/) : Session pour la gestion de l'utilisateur connecté

B. Diagramme de classes

Les classes principales du système :

- Produit : id, nom, categorie, quantite, unite, seuilAlerte, prixUnitaire, dateExpiration, fournisseur
- Mouvement : id, produitId, type (ENTREE/SORTIE), quantite, motif, dateHeure, utilisateur
- DatabaseManager : singleton gérant la connexion PostgreSQL et l'initialisation des tables
- ProduitDAO : findAll(), save(), update(), delete(), findStockFaible(), valeurTotaleStock()
- MouvementDAO : save(), findByProduit(), findRecent(), countMouvementsAujourd'hui()

C. Modèle de base de données

Trois tables principales :

Table	Colonnes principales	Description
produits	id, nom, categorie, quantite, unite, seuil_alerte, prix_unitaire	Catalogue de tous les produits
mouvements	id, produit_id, type, quantite, motif, date_heure, utilisateur	Historique entrées/sorties
utilisateurs	id, nom, prenom, login, mot_de_passe, role	Comptes admin et employé

D. Charte graphique

- Thème sombre (dark mode) : fond #0F1117, cartes #161B22
- Couleur principale : bleu GitHub (#1F6FEB) — boutons et accents
- Couleur alerte : rouge (#F85149) — stocks faibles
- Couleur succès : vert (#3FB950) — stocks corrects

- Typographie : Police système Java, tailles 11-24px selon le contexte

Partie 4 — Développement

A. Outils de développement

Catégorie	Outil / Technologie
Langage	Java 17
Framework UI	JavaFX 21
Base de données	PostgreSQL
Gestionnaire dépendances	Apache Maven
IDE	IntelliJ IDEA
Administration BDD	pgAdmin
Modélisation	StarUML
Versioning	Git

B. Fonctionnalités développées

1. Tableau de bord (DashboardView)

Le tableau de bord affiche quatre KPIs en temps réel : nombre total de produits référencés, nombre d'alertes stock actives, valeur totale du stock valorisé en roupies mauriciennes, et nombre de mouvements enregistrés dans la journée. Deux widgets complètent la vue : la liste des produits en alerte et les six derniers mouvements.

2. Gestion des produits (StockView)

La vue stock affiche la liste complète des 54 produits dans un TableView JavaFX avec colonnes : nom, catégorie, quantité (colorée en rouge si sous le seuil), seuil d'alerte, prix unitaire, fournisseur et statut (badge OK/Faible). Un champ de recherche et un filtre par catégorie permettent de retrouver rapidement tout produit. Les actions Modifier et Supprimer sont accessibles directement dans chaque ligne.

3. Entrées et Sorties (MouvementView)

Un formulaire permet d'enregistrer chaque entrée (achat, livraison) ou sortie (consommation, vente) en sélectionnant le produit, la quantité et un motif. Chaque mouvement est horodaté automatiquement et associé à l'utilisateur connecté. La quantité en stock est mise à jour instantanément.

4. Alertes stock (AlertesView)

La vue alertes liste uniquement les produits dont la quantité est inférieure ou égale au seuil d'alerte défini. Chaque ligne affiche le nom, la catégorie, la quantité actuelle et le seuil, avec une mise en évidence visuelle en rouge pour faciliter la lecture.

5. Historique (HistoriqueView)

L'historique présente tous les mouvements enregistrés dans un tableau filtrable par type (entrée/sortie), par produit et par période. Chaque ligne affiche la date/heure, le type de mouvement (badge coloré), le produit, la quantité et le motif.

6. Authentification et rôles

Deux comptes sont préconfigurés : Admin (accès complet) et Employé cuisine (accès restreint aux mouvements). La session est gérée via la classe Session.java, accessible depuis toutes les vues.

C. Données préremplies

La base de données est initialisée automatiquement au premier lancement avec 54 produits réels du restaurant, répartis en 8 catégories :

Catégorie	Nb produits	Exemples
Viandes	5	Poulet, agneau, bœuf, saucisse
Fruits de mer	4	Crevettes, calamar, crabe, poisson
Féculents & Boulangerie	6	Riz, mine, mee-foon, pain
Légumes	8	Oignons, tomates, ail, pak-choi
Épicerie & Condiments	9	Huile, sauce soja, sel, poivre
Boissons	13	Gazeuses, eaux, jus, canettes
Fournitures	9	Barquettes, sacs, bols take-away

Partie 5 — Recettage, Déploiement et Documentation

A. Recettage — Tests fonctionnels

Un test fonctionnel complet a été effectué avec le responsable du restaurant en semaine 5 :

Fonctionnalité testée	Résultat	Statut
Connexion utilisateur	Login admin et employé OK	✓ Validé
Ajout d'un produit	Enregistrement correct en BDD	✓ Validé
Modification d'un produit	Mise à jour correcte	✓ Validé
Suppression d'un produit	Suppression avec confirmation	✓ Validé
Enregistrement entrée	Stock mis à jour instantanément	✓ Validé
Enregistrement sortie	Stock mis à jour instantanément	✓ Validé
Alerte stock faible	Produits en rouge détectés	✓ Validé
Historique mouvements	Filtres fonctionnels	✓ Validé
Dashboard KPIs	Valeurs correctes en temps réel	✓ Validé

L'ensemble des fonctionnalités a été validé par le responsable du restaurant lors du test de recette.

B. Mise en place

Déploiement en local

L'application a été déployée sur le poste principal du restaurant. PostgreSQL a été installé et configuré localement. La base de données kotrobert_stock est initialisée automatiquement au premier lancement, avec création des tables et insertion des 54 produits de démarrage.

Prérequis d'installation

- Java 17 ou supérieur installé sur le poste
- PostgreSQL installé et démarré
- Fichier config.properties configuré avec les identifiants de la base
- Lancement via IntelliJ IDEA ou exécutable JAR

C. Documentation technique

- Guide d'installation : prérequis Java et PostgreSQL, configuration du fichier config.properties

- Manuel utilisateur : description des modules avec captures d'écran
- Schéma de la base de données : tables produits, mouvements, utilisateurs
- Javadoc des classes DAO et modèles

D. Formation

Une session de formation d'une demi-journée a été organisée avec le responsable et l'employé cuisine :

- Prise en main de l'interface et navigation entre les modules
- Enregistrement d'une entrée et d'une sortie de produit
- Consultation des alertes et de l'historique
- Ajout et modification d'un produit

Conclusion

Ce stage de six semaines au restaurant Kot Robert à l'Île Maurice a été une expérience très enrichissante sur le plan technique et professionnel.

Sur le plan technique, j'ai pu approfondir mes compétences en développement Java avec le framework JavaFX pour la création d'interfaces graphiques desktop, en conception et manipulation de bases de données PostgreSQL, en architecture MVC et en pattern DAO. La gestion de projet avec Maven et IntelliJ IDEA m'a également permis de travailler dans un environnement de développement professionnel.

Sur le plan professionnel, ce stage m'a permis de comprendre les besoins concrets d'un petit commerce en matière de gestion informatique, d'interagir directement avec le client pour affiner les fonctionnalités, et de livrer une solution opérationnelle répondant à un besoin réel.

L'application développée est aujourd'hui utilisée quotidiennement par le personnel du restaurant pour suivre les stocks, enregistrer les mouvements et anticiper les ruptures grâce au système d'alertes. Le responsable a exprimé sa satisfaction lors de la livraison finale.

Ce deuxième stage, combiné au premier effectué à la SMMC Port Toamasina, m'a permis de travailler sur deux environnements et deux technologies différentes (PHP/MySQL pour le web et Java/PostgreSQL pour le desktop), renforçant ainsi ma polyvalence en tant que développeur.

Annexes

Annexe 1 — Structure du projet

- src/main/java/com/kotrobert/model/ — Classes Produit et Mouvement
- src/main/java/com/kotrobert/dao/ — DatabaseManager, ProduitDAO, MouvementDAO
- src/main/java/com/kotrobert/ui/ — MainApp, DashboardView, StockView, MouvementView, AlertesView, HistoriqueView
- src/main/java/com/kotrobert/util/ — Session
- src/main/resources/config.properties — Configuration de la base de données
- pom.xml — Dépendances Maven (JavaFX 21, PostgreSQL JDBC)

Annexe 2 — Captures d'écran de l'application

(Insérer ici les captures d'écran : tableau de bord, liste des produits, formulaire ajout produit, vue alertes, historique des mouvements)

Annexe 3 — Bibliographie et Webographie

- Documentation officielle Java 17 : <https://docs.oracle.com/en/java/javase/17/>
- Documentation JavaFX 21 : <https://openjfx.io/javadoc/21/>
- Documentation PostgreSQL : <https://www.postgresql.org/docs/>
- Maven Repository : <https://mvnrepository.com/>
- OpenClassrooms — Développez des applications Java avec JavaFX
- Baeldung — Guide complet sur les DAO en Java

Annexe 4 — Glossaire

Terme	Définition
JavaFX	Framework Java pour le développement d'interfaces graphiques desktop
PostgreSQL	Système de gestion de base de données relationnelle open source
MVC	Modèle-Vue-Contrôleur — patron d'architecture logicielle
DAO	Data Access Object — couche d'abstraction pour l'accès aux données
Maven	Outil de gestion de dépendances et de build pour Java
JDBC	Java Database Connectivity — API Java pour se connecter aux bases de données
Singleton	Patron de conception garantissant une instance unique d'une classe
Take-away	Vente de plats à emporter

